

CSI v10 — Creating Reports Using Mongoose

Study notes from the Infor official training workbook (166 pages)

Why this guide matters

Standard CSI reports are built inside the Mongoose framework using **Report Type forms**, **FlexLayout** components, and **DataViews** (IDO methods that supply data to a report). This is a distinctly different approach from SQL Server Reporting Services (SSRS) — Mongoose reports live in WinStudio and deploy with the application metadata.

This guide is essential if you need to: build a new operational report, modify an existing Mongoose report, understand why a report runs slowly, or add parameters/criteria to an existing report.

1. Mongoose Reporting Architecture

Mongoose reports are WinStudio forms of a special type — **Report Type forms**. They display and format data but do not allow editing. The data pipeline is:

Stage	What happens
1. Report Criteria form	User enters parameters (date ranges, item filters, etc.).
2. DataView / IDO Method called	The Report Type form calls one or more IDO methods (DataViews) that query the database and return result sets.
3. Report Type form renders	The FlexLayout component organizes the result set into a formatted, printable layout.
4. Output	User previews, prints, or exports (PDF, Excel, CSV).

2. IDO Methods and DataViews

A **DataView** in Mongoose reporting is simply an IDO method (usually of type Fetch) whose result set drives a report. The method accepts parameters (criteria) and returns a structured set of rows and columns.

Creating a DataView IDO method

- On the IDO, add a new method of type Fetch.
- The method body is a stored procedure that accepts @parameters and returns a result set.
- Define the method's parameters: name, direction (input/output), data type.
- The result set's column names become the property names available for binding in the report.

Key rules for DataView stored procedures

- Always use a SELECT ... FROM ... WHERE pattern that returns a consistent column set.
 - Add NOLOCK hints on read-heavy reports to avoid blocking production transactions.
 - Use parameters for all filter criteria — never hard-code values in the procedure.
 - Test the stored procedure directly in SQL Server Management Studio before wiring it to a report.
-

3. Report Type Forms

A Report Type form is a WinStudio form with its Form Type property set to **Report**. This activates the report-specific toolset in Design Mode.

Property / Feature	What it controls
Form Type = Report	Activates the FlexLayout toolbox and report-specific behaviors.
Collection binding	The form's collection is bound to a DataView IDO method (not a standard IDO).
Report Criteria link	A separate 'criteria' form is linked via the Report's Criteria Form Name property. The criteria form collects parameters that are passed to the DataView.
Page setup	Paper size, orientation, margins — set via Form sheet > Page Setup properties.
Preview / Print / Export	Standard toolbar buttons. Output formats: PDF, Excel, CSV, directly to a printer.

Finding existing reports: In the Explorer, reports appear under their module (e.g., Modules > Customer > Reports). You can also use Ctrl+O and search by form name.

4. FlexLayout — The Report Layout Component

FlexLayout is the specialized layout component used inside Report Type forms. It provides bands (sections) for organizing report content.

FlexLayout Band	Purpose
Report Header	Printed once at the very top of the report (title, logo, run date, parameters used).
Page Header	Printed at the top of every page (column headings, page number).
Group Header	Printed once at the start of each group (e.g., 'Customer: Acme Corp').
Detail Band	Printed once per data row — the main body of the report.
Group Footer	Printed at the end of each group (subtotals, group summaries).
Page Footer	Printed at the bottom of every page (page X of Y, footer text).
Report Footer	Printed once at the very end (grand totals, summary statistics).

Adding components to FlexLayout bands

- Drag standard WinStudio components (Static, Edit, Grid, etc.) into a band.
 - Bind components to DataView properties using the standard Binding property.
 - Use Static components with calculated expressions for running totals, formatted dates, page numbers.
 - **SUM / COUNT / AVG functions** are available in calculated field expressions for group and report footers.
-

5. Secondary Collections

A Secondary Collection adds a child data set to a report — for example, order lines under each order header. It is a second IDO method (DataView) that is linked to the primary collection via a join key.

Concept	Detail
Primary collection	The main DataView that drives the report (e.g., Customer Orders).
Secondary collection	A child DataView (e.g., Order Lines) linked to the primary via a common key (e.g., CoNum).
Link property	Set on the secondary collection: which property in the child matches which property in the parent.
Rendering	The Detail Band can contain a nested FlexLayout bound to the secondary collection, printing child rows under each parent row.

Performance note: Each parent row causes a separate query for child rows unless you pre-join in the DataView stored procedure. For large data sets, consider returning everything in one query and using GROUP BY in the report.

6. Sub-Reports

A Sub-Report embeds one complete Report Type form inside another. Use sub-reports when two datasets are too different to join in SQL but need to appear together on the printed page.

Concept	Detail
Parent report	The outer report. Contains a Sub-Report component in one of its bands.
Child report	A fully independent Report Type form, designed separately.
Linking	Pass parameters from the parent to the child via the Sub-Report component's parameter bindings (parent property → child parameter).
Rendering	The child report renders inline at the position of the Sub-Report component.
When to use vs. Secondary Collection	Use Secondary Collection for simple parent-child row nesting. Use Sub-Report when the child data has a completely different layout, grouping, or criteria logic.

7. Report Criteria Forms

A Report Criteria form is a standard WinStudio form that collects user input (date ranges, filters, checkboxes) before the report runs. It is linked to the Report Type form via the Report's **Criteria Form Name** property.

Element	How it works
Parameter fields	Standard Edit/DateCombo/Combo components on the criteria form, not bound to any IDO — just collecting input.
Run button	A button that reads the criteria field values and passes them as parameters to the Report Type form's DataView IDO method.

Element	How it works
Default values	Set via form event handlers: on FormLoad, set default date ranges, default filter values, etc.
Validation	Validate criteria before calling the report (e.g., ensure From Date ≤ To Date).

Access pattern: The user opens the Criteria form, fills in parameters, clicks Run (or Preview). The Criteria form calls the Report Type form with parameters. The Report Type form calls its DataView. Data is returned and rendered. The user then uses Preview → Print or Export.

8. IDO Collections on Reports — Using Standard IDO Data

In addition to DataView methods, a Report Type form can bind to a standard IDO's LoadCollection — effectively treating any existing IDO as a report data source without writing a custom DataView method.

- Bind the report's collection directly to an existing IDO (e.g., SLCustomers).
- Set a Permanent Filter Expression on the report's collection to restrict data.
- Use this approach for simple reports where the IDO already returns the right data and no complex SQL joins are needed.
- For complex multi-table joins or aggregations, a custom DataView IDO method (stored procedure) will always give better performance and flexibility.

Quick Reference: Report Development Checklist

Step	Action
1. Design the data	Write and test the stored procedure in SQL Server Management Studio first.
2. Create the IDO method (DataView)	Add a Fetch method to an appropriate IDO; reference the stored procedure; define parameters.
3. Create the Criteria form	Standard WinStudio form — collect user parameters; wire Run button.
4. Create the Report Type form	New form, Form Type = Report; bind to DataView; set Criteria Form Name.
5. Build FlexLayout	Add bands; bind components to DataView properties; add group headers/footers if needed.
6. Add Secondary Collections	If child rows needed: define child DataView; link to parent; add nested FlexLayout.
7. Add Sub-Reports	If independent child layouts needed: build child report separately; add Sub-Report component in parent.
8. Test end-to-end	Open Criteria form → enter parameters → run → verify data accuracy → test edge cases (empty result, large result).

Step	Action
9. Register in Explorer	Add the Criteria form to the appropriate module folder in the Explorer navigation.